

Resiliency in Distributed Systems

Two chapters from the book Understanding Distributed Systems by Roberto Vitillo



GERGELY OROSZ

SEP 28, 2022 • PAID



35



2



Share



Understanding the ins and outs of distributed systems for engineers and for anyone who wants to build systems that mean systems with high load, high availability, and a large amount of data, or ones built for scale. These are pretty common across both front-end and back-end systems.

One of the most interesting books I've read is [Understanding Distributed Systems](#). The book was written by Roberto Vitillo, a Principal Engineer at Mozilla, then a Principal Engineer at Uber, and was released in February of this year.

The book is structured in five parts:

1. **Communication.** Reliability, consistency, and availability.
2. **Coordination.** System replication, coordination avoidance, and consistency.
3. **Scalability.** HTTP caching, content delivery networks, partitioning, file storage, data storage, caching, microservices, control planes and data planes, messaging.
4. **Resiliency.** Common failure causes, redundancy, fault isolation, downstream resiliency, upstream resiliency.
5. **Maintainability.** Testing, continuous delivery and deployment, monitoring, observability, and manageability.



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: [pragmaticurl.com/books](#). Advisor at mobile.dev



The Pragmatic Engineer

By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

ackend
ems can
ge
ms are

[distributed](#)
ineer at
k was

plication,

I like how the book works its way from the theory needed to understand distributed systems - communication and coordination - to practical topics like scalability and resiliency. The book closes with topics on maintainability, which is an area I found surprisingly little focus with most books.

I reached out to Roberto asking if he'd be open to sharing a few chapters of the book with newsletter readers, and Roberto agreed to do so. I chose two chapters on resiliency, from Part 4. If you'd like to dive deeper into other topics, you can buy the [e-book on Roberto's website](#), or the [print book off Amazon](#).

In this excerpt, we cover:

1. Downstream resiliency

- Timeout
- Retry: exponential backoff
- Circuit breaker

2. Upstream resiliency

- Load shedding
- Load leveling
- Rate limiting: single process
- Constant work



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



The Pragmatic Engineer

By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

Note that - as always - no link

been paid

to endorse or recommend this book. More in [my ethics statement](#).

1. Downstream resiliency (Chapter 27)

Now that we have discussed how to reduce the impact of faults at the architectural level with redundancy and partitioning, we will dive into tactical resiliency patterns that stop faults from propagating from one component or service to another. In this chapter,

we will discuss patterns that protect a service from failures of downstream dependencies.

Timeout

When a network call is made, it's best practice to configure a timeout to fail the call if no response is received within a certain amount of time. If the call is made without a timeout, there is a chance it will never return, and as mentioned in chapter 24, network calls that don't return lead to resource leaks. Thus, the role of timeouts is to detect connectivity faults and stop them. In general, timeouts are a must when acquiring a mutex.

Unfortunately, some network libraries have no timeout while others have no timeout. XMLHttpRequest is the web's default timeout is zero, which

```
var xhr = new XMLHttpRequest();
xhr.open("GET", "/api/endpoint");
// No timeout
// be set explicitly
xhr.timeout = 5000;
xhr.onload = function() {
    // Request finished
};
xhr.ontimeout = function (e) {
    // Request timed out
};
xhr.send(null);
```



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



The Pragmatic Engineer

By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

The fetch web API is a modern replacement for XMLHttpRequest that uses Promises. When the fetch API was initially introduced, there was no way to set a timeout at all. Browsers have only later added support for timeouts through the Abort API. Things aren't much rosier for Python; the popular requests library uses a default timeout of infinity. And Go's HTTP package doesn't use timeouts by default.

Modern HTTP clients for Java and .NET do a better job and usually, come with default timeouts. For example, .NET Core HttpClient has a default timeout of 100 seconds. It's lax but arguably better than not having a timeout at all.

As a rule of thumb, always select third-party libraries that make

But how do we determine a good false timeout rate. For example, are we willing to accept that 0.1% of requests will time out (i.e., 0.1% of the timeout based on the 99.

We also want to have good metrics for network call, like the duration of the timeout was triggered. The point I want to make here is that we need to monitor the points of our systems, or we will have issues.

Ideally, a network call should

and monitors the request so that we don't have to remember to do this for each call. Alternatively, we can also use a reverse proxy co-located on the same machine, which intercepts remote calls made by our process. The proxy can enforce timeouts and monitor calls, relieving our process of this responsibility. We talked about this in section 18.3 when discussing the sidecar pattern and the service mesh.

Retry



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



The Pragmatic Engineer

By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

We know by now that a client should configure a timeout when making a network request. But what should it do when the request fails or times out? The client has two options at that point: it can either fail fast or retry the request. If a short-lived connectivity issue caused the failure or timeout, then retrying after some backoff time has a high probability of succeeding. However, if the downstream service is overwhelmed, retrying immediately after will only worsen matters. This is why retrying needs to be slowed down with increasingly longer delays between the individual retries until either a maximum number of retries is reached or enough time has passed since the initial request.

Exponential backoff

To set the delay between retries, the delay is derived by multiplying exponentially after each attempt:

$$\text{delay} = n \times \text{base_delay}$$

For example, if the cap is set to 10, then the first retry delay is 2 seconds, and any further delay will be capped at 10 seconds.

Although exponential backoff helps with the dependency, it still has a problem: when the downstream service is temporarily degraded, multiple clients will likely see their requests failing around the same time. This will cause clients to retry simultaneously, hitting the downstream service with load spikes that further degrade it, as shown in Figure 27.1.



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



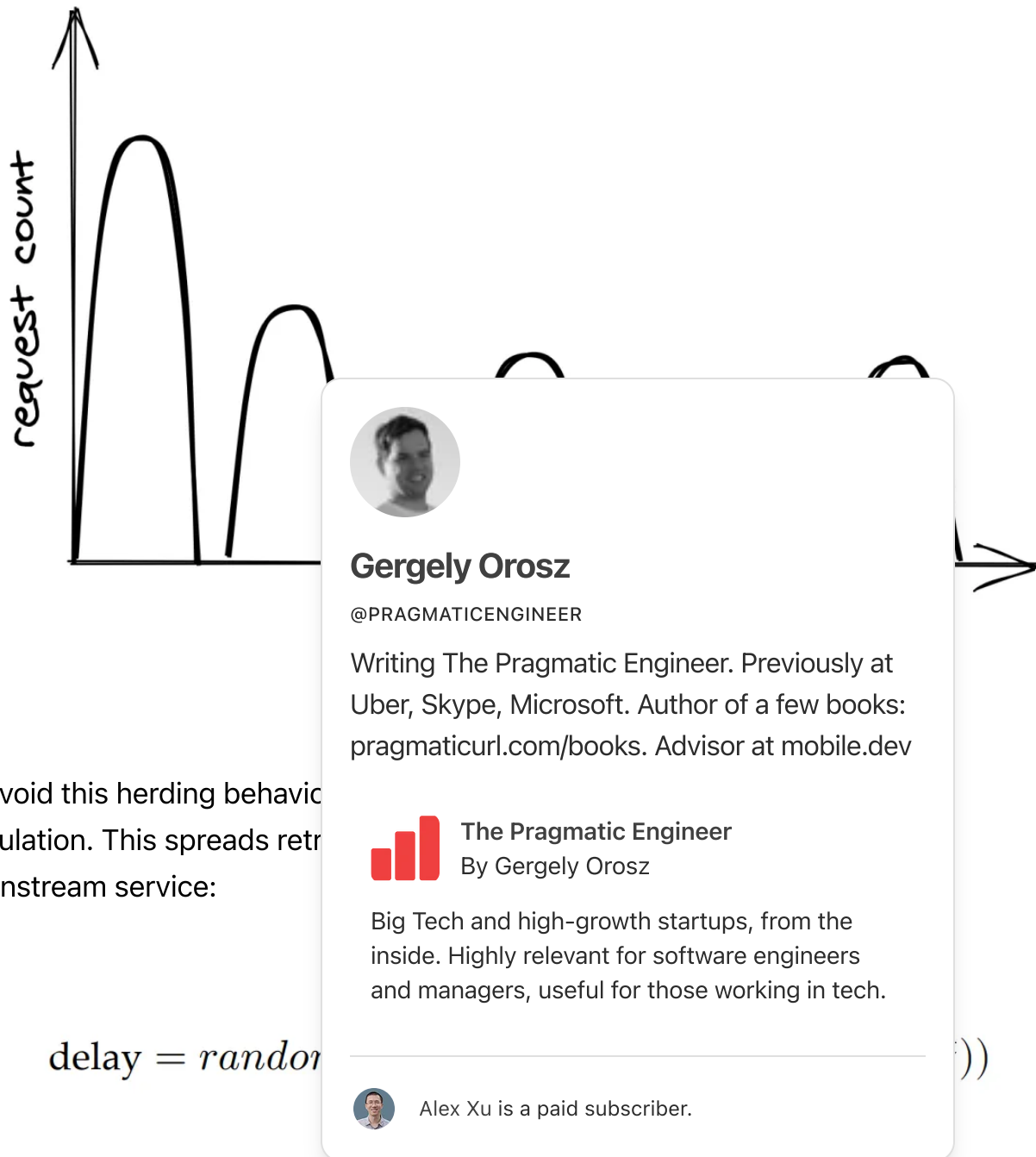
The Pragmatic Engineer

By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.



To avoid this herding behavior, a random delay calculation. This spreads retries across the downstream service:

$\text{delay} = \text{random}$

Actively waiting and retrying failed network requests isn't the only way to implement retries. In batch applications that don't have strict real-time requirements, a process can park a failed request into a retry queue. The same process, or possibly another, can read from the same queue later and retry the failed requests.

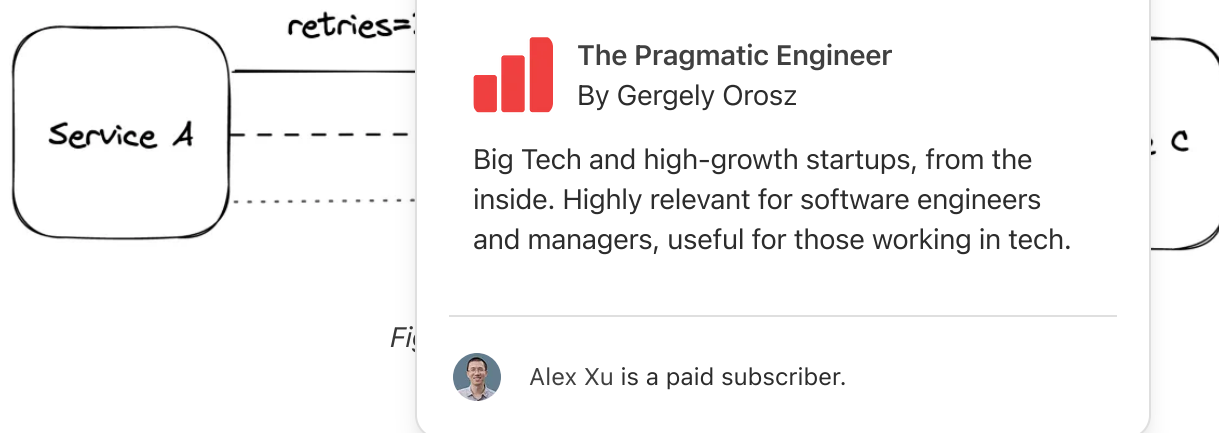
Just because a network call can be retried doesn't mean it should be. If the error is not short-lived, for example, because the process is not authorized to access the remote endpoint, it makes no sense to retry the request since it will fail again. In this case, the

process should fail fast and cancel the call right away. And as discussed in chapter 5.7, we should also understand the consequences of retrying a network call that isn't idempotent and whose side effects can affect the application's correctness.

Retry amplification

Suppose that handling a user request requires going through a chain of three services. The user's client calls service A, which calls service B, which in turn calls service C. If the intermediate request from service B to service C fails, should B retry the request or not? Well, if B does retry it, A will perceive a longer execution time for its request, making it more likely for the client to hit A's timeout, making it more likely for the client to hit A's timeout.

Having retries at multiple levels of a dependency chain leads to a snowball effect of retries — the deeper a service is in the chain, the more retries are exposed to due to retry amplification (see Figure 5.1).



And if the pressure gets bad enough, this behavior can easily overload downstream services. That's why, when we have long dependency chains, we should consider retrying at a single level of the chain and failing fast in all the others.

Circuit breaker

Suppose a service uses timeouts to detect whether a downstream dependency is unavailable and retries to mitigate transient failures. If the failures aren't transient and the downstream dependency remains unresponsive, what should it do then? If the



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



The Pragmatic Engineer
By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

service keeps retrying failed requests, it will necessarily become slower for its clients. In turn, this slowness can spread to the rest of the system.

To deal with non-transient failures, we need a mechanism that detects long-term degradations of downstream dependencies and stops new requests from being sent downstream in the first place. After all, the fastest network call is the one we don't have to make. The mechanism in question is the circuit breaker, inspired by the same functionality implemented in electrical circuits.

The goal of the circuit breaker is to allow a sub-system to fail without slowing down the caller. To protect the system, the circuit breaker will stop sending requests to the sub-system. Later, when the sub-system is healthy again, the circuit breaker will allow requests to go through again.

Unlike retries, circuit breaker is particularly useful for non-transient failures. The expectation is that the next call will succeed. The expectation is that the next call will succeed.

A circuit breaker can be implemented in closed, and half-open (see Fig. 1).



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



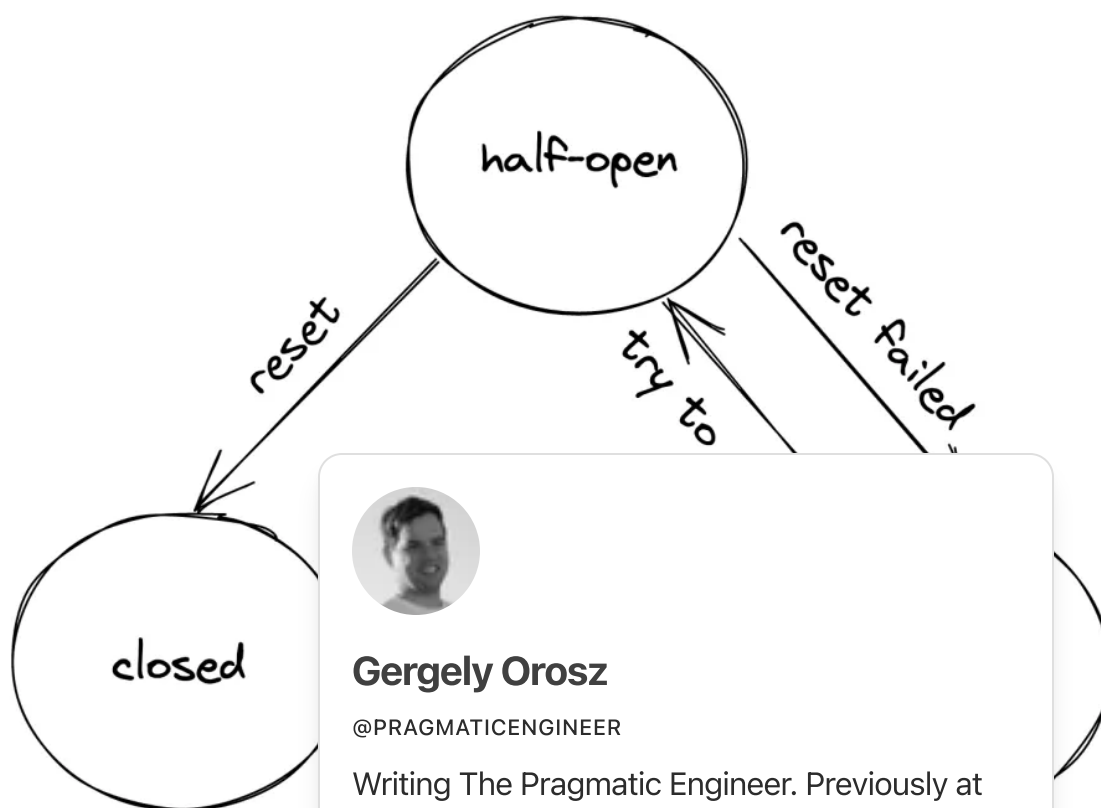
The Pragmatic Engineer

By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.



Fig

**Gergely Orosz**

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev

**The Pragmatic Engineer**

By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

In the closed state, the circuit
In this state, the circuit break
the number goes over a certa
breaker trips and opens the c

work calls.
timeouts. If
e circuit

When the circuit is open, net
open circuit breaker can have

As an
it should

happen when a downstream dependency is down. If the dependency is non-critical, we want our service to degrade gracefully rather than to stop entirely. Think of an airplane that loses one of its non-critical sub-systems in flight; it shouldn't crash but rather gracefully degrade to a state where the plane can still fly and land. Another example is Amazon's front page; if the recommendation service is unavailable, the page renders without recommendations. It's a better outcome than failing to render the whole page entirely.

After some time has passed, the circuit breaker gives the downstream dependency another chance and transitions to the half-open state. In the half-open state, the next call is allowed to pass through to the downstream service. If the call succeeds, the circuit breaker transitions to the closed state; if the call fails instead, it transitions back to the open state.

You might think that's all there is to understand how a circuit breaker works, but the devil is in the details. For example, how many failures are "enough to consider a downstream dependency down? How long should the circuit breaker wait to transition from the open to the half-open state? Only by using data about past failures

Upstream resiliency

The previous chapter discussed failures, like failures to reach a downstream service, and discuss mechanisms to protect against them.

Load shedding

A server has very little control over the load it receives. The operating system has a mechanism to reject requests when reached, causes new connections to be refused. Typically, under extreme load, a server will reject requests. It runs out of resources like memory or CPU, and its response time to increase until it becomes unresponsive to the outside world.

When a server operates at capacity, it should reject excess requests so that it can dedicate its resources to the requests it's already processing. For example, the server could use a counter to measure the number of concurrent requests being processed that is incremented when a new request comes in and decreased when a response is sent. The server can then infer whether it's overloaded by comparing the counter with a threshold that approximates the server's capacity.



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



The Pragmatic Engineer
By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

When the server detects that it's overloaded, it can reject incoming requests by failing fast and returning a response with status code 503 (Service Unavailable). This technique is also referred to as load shedding. The server doesn't necessarily have to reject arbitrary requests; for example, if different requests have different priorities, the server could reject only low-priority ones. Alternatively, the server could reject the oldest requests first since those will be the first ones to time out and be retried, so handling them might be a waste of time.

Unfortunately, rejecting a request doesn't completely shield the server from the cost of handling it. Depending on how the server handles the request, it might still have to pay the price of opening a connection, parsing the request, and so on. Hence, load shedding can only reduce the cost of rejecting requests will eventually be rejected.

Load leveling

There is an alternative to load shedding: the server can expect a prompt response. The server can buffer requests from clients and the service. The server can process requests up to its capacity, allowing it to process requests as they come in.

This pattern is referred to as load leveling. When the server receives spikes, which the channel smooths out, the server can catch up eventually, a large backlog of requests will be processed as discussed in chapter 23.



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



The Pragmatic Engineer
By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

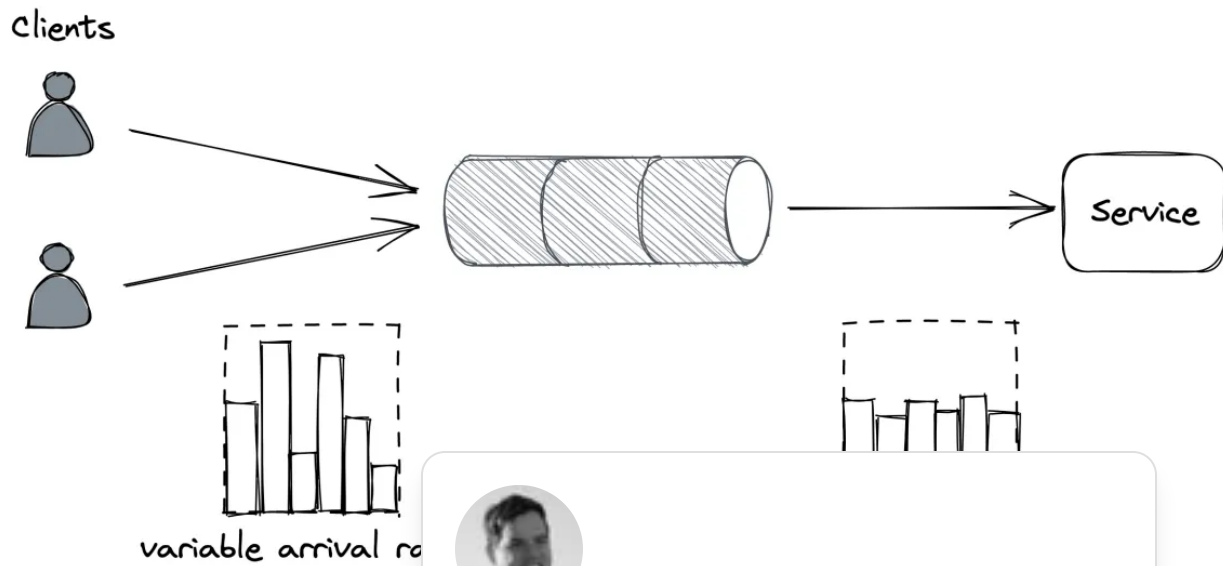


Figure 28.1: The ch

Load-shedding and load leveling protect a service from getting be scaled out. This is why the auto-scaling, which detects t its scale to handle the additio

Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



The Pragmatic Engineer
By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

Rate-limiting

Rate-limiting, or throttling, is is exceeded. A service can ha bytes received within a time i keys, or IP addresses.

t rather
needs to
ed with
increases

ific quota
ests or
users, API

For example, if a service with a quota of 10 requests per second per API key receives on average 12 requests per second from a specific API key, it will, on average, reject 2 requests per second from that API key.

When a service rate-limits a request, it needs to return a response with a particular error code so that the sender knows that it failed because a quota has been exhausted. For services with HTTP APIs, the most common way to do that is by returning a

response with status code 429 (Too Many Requests). The response should include additional details about which quota has been exhausted and by how much; it can also include a Retry-After header indicating how long to wait before making a new request:

If the client application plays by the rules, it will stop hammering the service for some time, shielding the service from non-malicious users monopolizing it by mistake. In addition, this protects against bugs in the clients that cause a client to hit a downstream service for one reason or another repeatedly.

Rate-limiting is also used to enforce pricing tiers: if users want to use more resources, they should also be willing to pay more. This is a common pattern for SaaS products. For example, you can have different tiers with different quotas.

You would think that rate-limiting helps, but it only partially protects against a distributed denial of service (DDoS) attack, from which you can't free yourself. The price of opening a TLS connection is high, and the price of reading the key is even higher. Although rate limiting helps reduce their impact, it does not solve the problem.

Economies of scale are the only way to handle multiple services behind one gateway. If behind it are attacked, the gateway is amortized across all services. Rejecting the traffic upstream is a good idea, but running



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



The Pragmatic Engineer
By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

Although rate-limiting has some similarities with load shedding, they are different concepts. Load shedding rejects traffic based on the local state of a process, like the number of requests concurrently processed by it; rate-limiting instead sheds traffic based on the global state of the system, like the total number of requests concurrently processed for a specific API key across all service instances. And because there is a global state involved, some form of coordination is required.

Single-process rate limiting implementation

The distributed implementation of rate-limiting is interesting in its own right, and it's well worth spending some time discussing it. We will start with a single-process implementation first and then extend it to a distributed one.

Suppose we want to enforce a quota of 2 requests per minute, per API key. A naive approach would be to use a doubly-linked list per API key, where each list stores the timestamps of the last N requests received. Whenever a new request comes in, an entry is appended to the list with its corresponding timestamp. Then, periodically, entries older than a minute are

By keeping track of the list's
comparing it with the quota.
API key, which quickly becom
number of requests received

To reduce memory consump
requirements. One way to do
example of 1 minute, and kee
each bucket (see Figure 28.2



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



The Pragmatic Engineer

By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

12.00

1 min bucket

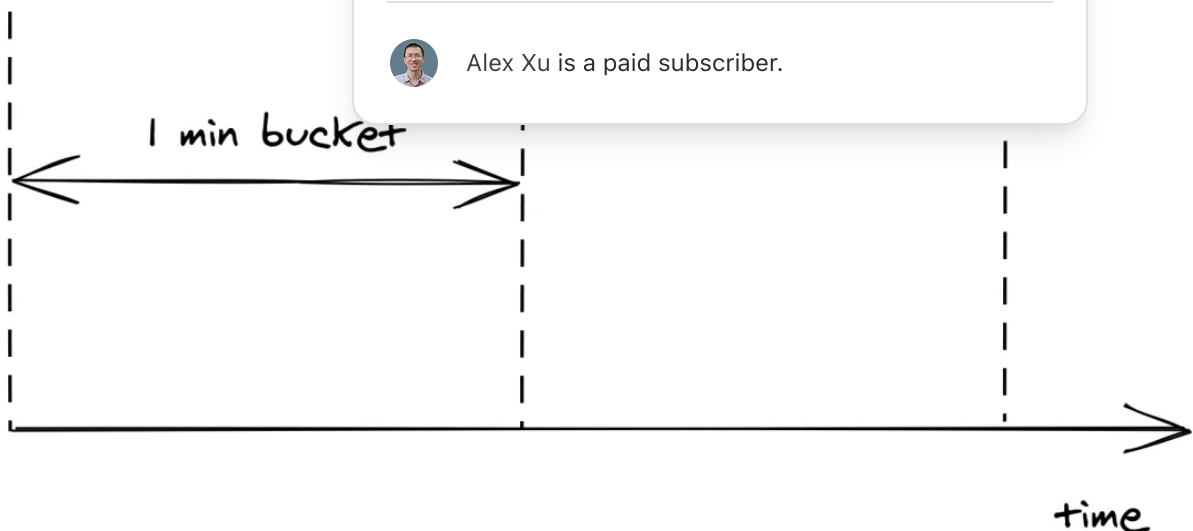


Figure 28.2: Buckets divide time into 1-minute intervals, which keep track of the number of requests seen.

A bucket contains a numerical counter. When a new request comes in, its timestamp is used to determine the bucket it belongs to. For example, if a request arrives at 12.00.18, the counter of the bucket for minute "12.00" is incremented by 1 (see Figure 28.3).



Figure 28.3: Wh

With bucketing, we can compute the number of requests seen in a way that doesn't grow with time. However, a bucket-friendly representation, how can we use it to implement rate-limiting? The idea is to use a sliding window that moves across the buckets in real time, keeping track of the number of requests within it.

The sliding window represents the interval of time used to decide whether to rate-limit or not. The window's length depends on the time unit used to define the quota, which in our case is 1 minute. But there is a caveat: a sliding window can overlap with multiple buckets. To derive the number of requests under the sliding window, we have to

compute a weighted sum of the bucket's counters, where each bucket's weight is proportional to its overlap with the sliding window (see Figure 28.4).

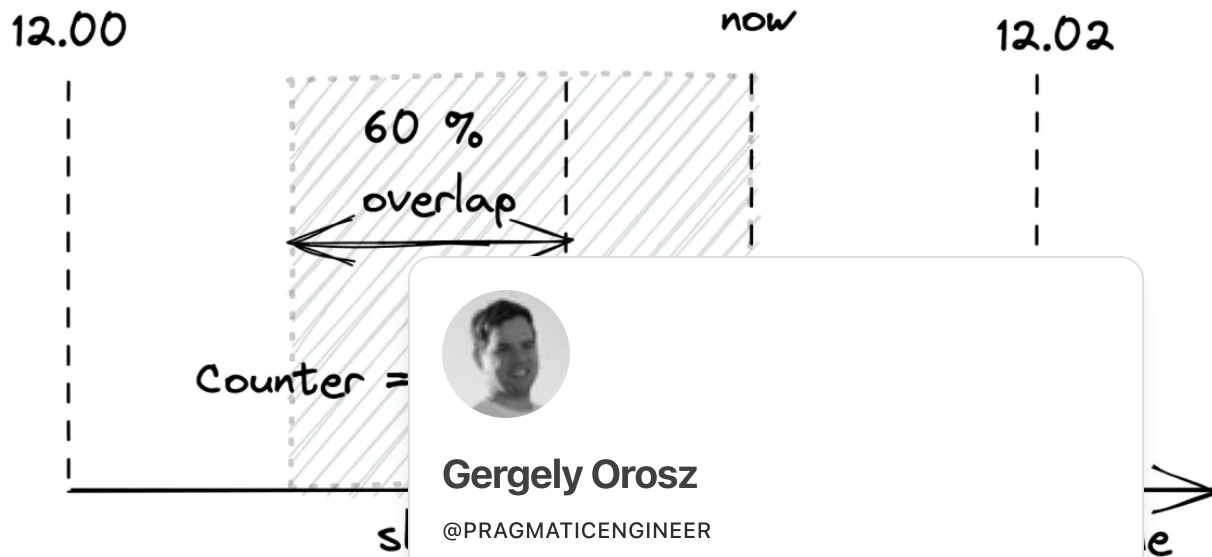


Figure 28.4: A b

Although this is an approximation, it can be made more accurate. For example, we can reduce the size of the 1-minute ones.

We only have to store as many buckets as there are in the given time. For example, with a 1-minute sliding window, the sliding window can overlap with at most 2 buckets. Thus, there is no point in storing the third oldest bucket, the fourth oldest one, etc.

To summarize, this approach requires two counters per API key, which is much more efficient in terms of memory than the naive implementation storing a list of requests per API key.

Distributed rate limiting implementation

When more than one process accepts requests, the local state is no longer good enough, as the quota needs to be enforced on the total number of requests per API key across all service instances. This requires a shared data store to keep track of the number of requests seen.

As discussed earlier, we need to store two integers per API key, one for each bucket. When a new request comes in, the process receiving it could fetch the current bucket, update it and write it back to the data store. But that wouldn't work because two processes could update the same bucket concurrently, which would result in a lost update. The fetch, update, and write back is a single transaction to avoid any race conditions.

Although this approach is fine for small-scale services, transactions are slow, and expensive. A shared data store would have to scale horizontally, as the data store is a hard dependency.

Let's address these issues. Rather than a get-and-increment operation, we can use a compare-and-increment operation. This can be emulated with a compare-and-swap operation, which has better performance than transactions.

Now, rather than updating the bucket updates in memory for each request, we can store it at the end of it (see Figure 1.10). This is a good trade-off as it reduces the number of requests sent to it.



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



The Pragmatic Engineer

By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

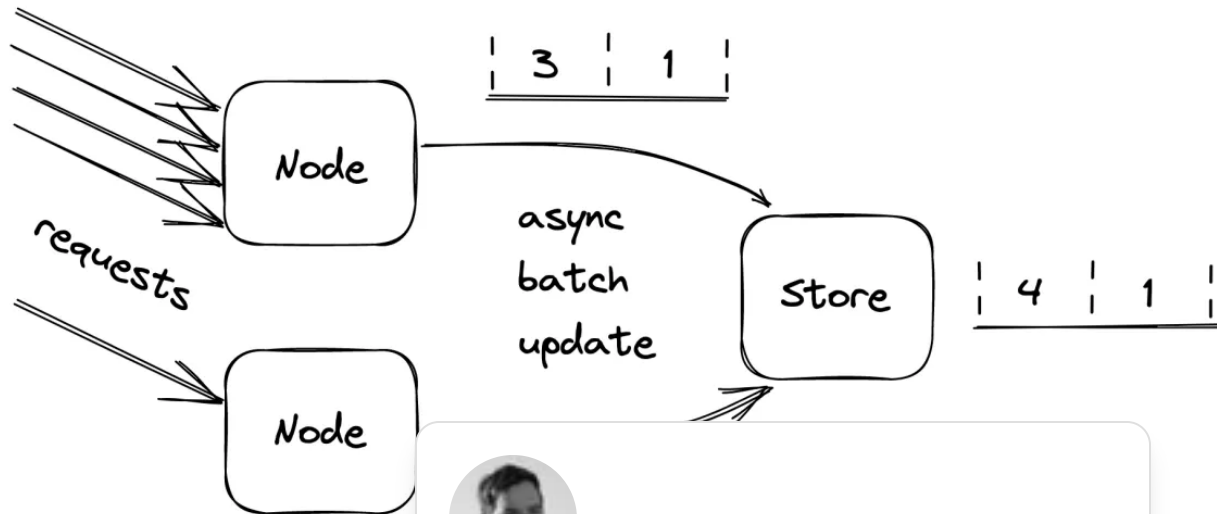


Figure 28.5: Server-side batching. Requests are batched and flushed to the store.

What happens if the data store is not available? If there is a network fault, we cannot write to the store. To maintain consistency and store state, we can buffer requests just because the data is not available. This can damage the business. Instead, we can read the state from the store.

Constant work

When overload, configuration changes, or other events occur differently from usual, we say the system is in a special mode.

These modes might trigger rare bugs, conflict with mechanisms that assume the happy path, and more generally make life harder for operators, since their mental model of how the application behaves is no longer valid. Thus, as a general rule of thumb, we should strive to minimize the number of modes.

For example, simple key-value stores are favored over relational databases in data planes because they tend to have predictable performance. A relational database has many operational modes due to hidden optimizations, which can change how specific



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



The Pragmatic Engineer
By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

queries perform from one execution to another. Instead, dumb key-value stores behave predictably for a given query, which guarantees that there won't be any surprises.

A common reason for a system to change behavior is overload, which can cause the system to become slower and degrade at the worst possible time. Ideally, the worst- and average-case behavior shouldn't differ. One way to achieve that is by exploiting the constant work pattern, which keeps the work per unit time constant.

The idea is to have the system perform the same amount of work under high load as under average load. And, if there is any variation under stress, it should be because the system is performing better, not worse. This is a different property from resilience, which is about surviving extreme load, while an antifragile one is about thriving under stress.

We have already seen one application of this in the propagation of configuration changes in chapter 22. For example, suppose a service stores a bag of settings for each data plane (to rate-limit requests). When a new plane needs to broadcast its settings, it sends an independent unit of work, the settings, to the number of changes.

If you don't see how this could be a problem, are updated for the majority of users (a business decision). This could lead to update messages to be sent to a data plane to handle them.



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



The Pragmatic Engineer
By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

The workaround to this problem is simple but powerful. The control plane can periodically dump the settings of all users to a file in a scalable and highly available file store like Azure Storage or AWS S3. The dump includes the configuration settings of all users, even the ones for which there were no changes. Data plane instances can then periodically read the dump in bulk and refresh their local view of the system's settings.

configuration. Thus, no matter how many settings change, the control plane periodically writes a file to the data store, and the data plane periodically reads it.

We can take this pattern to the extreme and pre-allocate empty configuration slots for the maximum number of supported users. This guarantees that as the number of users grows, the work required to propagate changes remains stable. Additionally, doing so allows to stress-test the system and understand its behavior, knowing that it will behave the same under all circumstances. Although this limits the number of users, a limit exists regardless of whether the constant work pattern is used or not. This approach is typically used in

well-defined maximum size a

The beauty of using the cons performs the same amount c have changed. This makes up writing and reading a large fil mechanism that only sends v

Another advantage of this ap thanks to its self-healing pro whatever reason, no harm is was pushed to all users by m and waiting it out. In contrast to implement correctly, as the corrupted updates.

To sum up, performing const work. Still, it's often worth considering it, given the increase in reliability and reduction in complexity it enables.



Gergely Orosz
@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



The Pragmatic Engineer
By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

cell has a s.

ally
n settings
periodically
complex

of faults
for
ity update
dump
uch harder
neal from

necessary

These were chapters 27 and 28 from the book Understanding Distributed Systems by Roberto Vitillo. To dive deeper, get the [e-book from the Understanding Distributed Systems website](#), or the [print book off Amazon](#). You can also follow Roberto on Twitter and connect with him [on LinkedIn](#).



35 Likes

2 Comments



Write a comment...

**Johnie Lee** Sep 28, 2022

One thing I don't see mentioned in the article is how distributed systems because of various reasons can be in an

It's important for the client to handle an indeterministic state. Just because you assume that the actual problem is determined, the failure has already happened.

This is why idempotency is important to alleviate a bunch of problems.

Also on the retry, if there is a way to know not to retry, downstream services

This was the root cause of the problem. The orchestrator knew that one of the downstream system (payment) was not working. This resulted in an agreement that services should not be retried but rather pass the error back to orchestrator to determine.

When you're working with distributed systems, things get a lot more complex. Rather than look at your individual service, you need to have a greater visibility on the end to end flow.

This is what separates a staff engineer from a junior engineer.

♡ LIKE (3) 💬 REPLY ⋮

1 reply by Gergely Orosz

**Gergely Orosz**

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev

**The Pragmatic Engineer**

By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.

1 more comment...

© 2023 Gergely Orosz · [Privacy](#) · [Terms](#) · [Collection notice](#)

Substack is the home for great writing



Gergely Orosz

@PRAGMATICENGINEER

Writing The Pragmatic Engineer. Previously at Uber, Skype, Microsoft. Author of a few books: pragmaticurl.com/books. Advisor at mobile.dev



The Pragmatic Engineer

By Gergely Orosz

Big Tech and high-growth startups, from the inside. Highly relevant for software engineers and managers, useful for those working in tech.



Alex Xu is a paid subscriber.